

HW2 第二次挑战：前缀缓存测量与 SGLang 请求流程

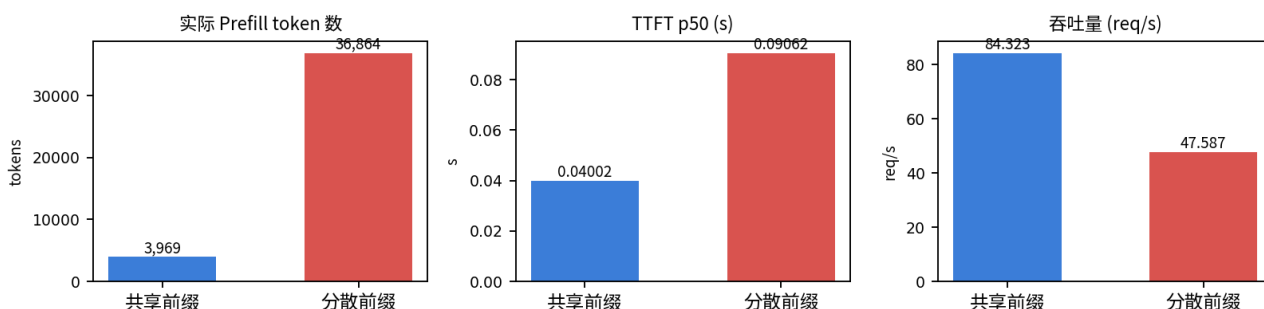
学号 0102603133 · SGLang 0.5.14 · Ray 2.56.0 · Qwen/Qwen3-0.6B · 4 × RTX 4090 D

1. 实验环境与方法

AutoDL 单节点 4 × RTX 4090 D (24 GB)，SGLang 0.5.14 (torch 2.11.0+cu130、transformers 5.8.1)，模型 Qwen/Qwen3-0.6B，RadixCache 保持默认开启。启动单个后端：python -m sglang.launch_server --model /root/autodl-tmp/models/Qwen3-0.6B --port 30000。测量脚本 src/target1/measure_prefix_cache.py：走原生 /generate 接口、直接发送 input_ids 并用流式响应解析每块 meta_info；两组各 32 条测量请求、最大并发 8，统一 temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。两条请求序列完全等长（1152 = 1024 前缀 + 128 后缀）：共享前缀组 32 条共用同一段 1024-token 前缀，分散前缀组每条前缀彼此无关。首次实验前先用短请求预热服务；每组测量前等待在飞请求结束、POST /flush_cache 确认成功（0.5.14 返回纯文本 Cache flushed），共享前缀组再发送 1 条预热请求把公共前缀载入 RadixCache；预热不计入结果。

2. 目标一：前缀缓存测量结果

指标	共享前缀组	分散前缀组	差异
成功请求 / 总数	32/32	32/32	一致
缓存命中率	89.23%	0.00%	+89.23 pp
命中 token 数	32,895	0	+32,895
实际 Prefill token 数	3,969	36,864	-32,895 (-89.2%)
吞吐量 (req/s)	84.32	47.59	+77.2%
TTFT p50 / p95 (s)	0.0400 / 0.0568	0.0906 / 0.1103	p50 -55.8%
TPOT p50 / p95 (s)	0.0032 / 0.0038	0.0054 / 0.0079	p50 -41.1%
端到端 p50 / p95 (s)	0.0922 / 0.1040	0.1633 / 0.1761	p50 -43.5%

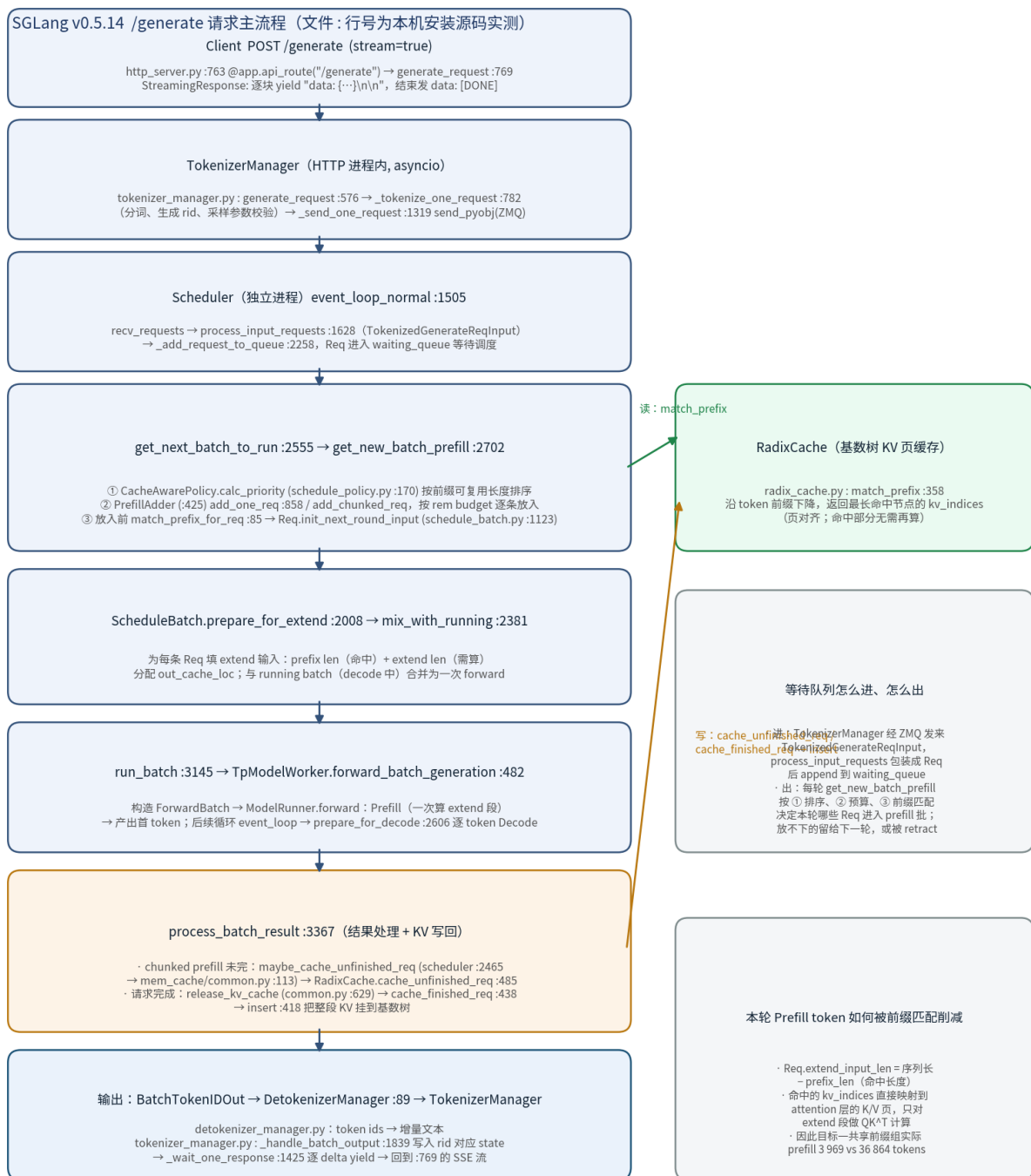


2.1 从因果自注意力解释前缀复用

自注意力为 $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$ 。因果解码器中位置 t 的查询 q_t 只与位置 $\leq t$ 的 K 、 V 作用（因果掩码把 $j > t$ 的分数置为 $-\infty$ ）。于是一旦前缀 $p_{1..m}$ 确定，它的 $K_{1..m}$ 、 $V_{1..m}$ 就与后续生成内容无关：权重矩阵逐层固定，位置编码只依赖位置本身而不依赖同批其它请求。所以相同 token、相同位置上的前缀 K/V 可被任何以该前缀开头的请求复用——RadixCache 按 token 前缀把 K/V 组织成基数树并共享节点。命中后仍需计算的只有未命中段：本实验共享前缀组每条 1152 tokens 中约 1024 命中（实测 89.23%），各自 128-token 后缀必须新算，实际 prefill 从 36 864 降到 3 969 tokens。

TTFT 与 TPOT 变化幅度不同，因为二者走不同路径：TTFT 主要由 prefill 计算量决定，前缀命中直接削减这条路径（p50 -55.8%）；TPOT 衡量首个 token 之后每 token 的解码时间，取决于 decode 批处理与显存带宽，与 prefill 无关，只因 prefill 对 decode 的抢占减少而小幅改善（0.00537 → 0.00317 s，-41.1%）。端到端延迟是两者之和，其改善（-43.5%）介于两者之间。

3. 任务二：SGLang v0.5.14 一条 /generate 请求的完整流程



3.1 流程说明 (对应流程图中①—④四个问题)

请求如何进入等待队列: HTTP 层 http_server.py:763 的 /generate 路由把请求交给

TokenizerManager.generate_request (tokenizer_manager.py:576), _tokenize_one_request (:782) 完成分词并生成 rid, _send_one_request (:1319) 用 ZMQ send_pyobj 把 TokenizedGenerateReqInput 发给 Scheduler 进程; Scheduler 在 event_loop_normal (scheduler.py:1505) 里 rcv_requests 收包, process_input_requests (:1628) 包装成 Req, _add_request_to_queue (:2258) 把它 append 进 waiting_queue, 等待每轮调度。

前缀匹配如何减少本轮 Prefill token: get_new_batch_prefill (scheduler.py:2702) 先用

CacheAwarePolicy.calc_priority (schedule_policy.py:170) 按「可复用前缀长度」给 waiting_queue 排序, 再由 PrefillAdder (:425) 按剩余 token/页预算逐条 add_one_req (:858); 放入前经 match_prefix_for_req (:85) → Req.init_next_round_input (schedule_batch.py:1123) →

RadixCache.match_prefix (radix_cache.py:358) 沿基数树下得到最长命中的 kv_indices。之后 prepare_for_extend (:2008) 把每条 Req 的输入拆成 prefix_len (命中, 直接引用已有 K/V 页) 与

extend_input_len（需新算）两段，注意力只对 extend 段做 QK^T ，本轮 prefill token 因此从整条序列缩到未命中部分。

新产生的 KV 如何写回缓存： chunked prefill 中未完成的请求在调度侧由 maybe_cache_unfinished_req (scheduler.py:2465 → mem_cache/common.py:113) 调用

RadixCache.cache_unfinished_req (:485)，把已完成段先挂进基数树供下一段续接；请求完成后 process_batch_result 走 release_kv_cache (common.py:629) → cache_finished_req (:438) → insert (:418)，把该请求完整的 KV 页插入基数树，成为后续请求可命中的前缀节点。

生成结果如何流式返回： Scheduler 把每步的 BatchTokenIDOut 发给

DetokenizerManager (detokenizer_manager.py:89) 增量解码成文本，再回到

TokenizerManager._handle_batch_output (:1839)，按 rid 追加到对应 state 的 out_list 并唤醒事件；generate_request 协程在 _wait_one_response (:1425) 里逐 delta yield，HTTP 层 (:769) 把这些块包成 StreamingResponse 的 SSE

帧「data:{...}\n\n」，最后发「data:[DONE]」。每个 delta 携带 meta_info（含 cached_tokens、prompt_tokens、completion_tokens），目标一的逐请求指标即从这条流上解析。

注：以上文件：行号均在本机安装的 sglang 0.5.14 源码中用 grep

定位核对 (site-packages/sglang/srt/...)，与流程图一致。任务二只关注主流程，未修改任何源码。